ORBIS SCHEDULER

Frontend Specification Document

Screen inventory, routing map, and component specification for the client application.

Hussain Jameel · ITA602 · Torrens University Australia

18 screens across 3 actors, mapped against all 29 backend endpoints

Purpose. Before scaffolding the client application, this document maps every screen it needs, one to one, against the actor it serves and the backend endpoint(s) it calls. Every row traces back to a use case already signed off and an endpoint already built and verified — nothing here introduces new backend scope. This mirrors the discipline already used for the API endpoint reference: design the full surface before writing implementation code.

Table of Contents

Screen Count Summary

Actor	Screens	Auth required	Backend groups covered
Public / Customer	6	None	Auth (4 routes) + Public (3 routes)
Business Owner	6	JWT + approved business	Business Profile, Availability, Forms & Fields, Bookings (17 routes)
Platform Admin	3	JWT + admin role	Admin (7 routes)
Total	**15**	—	**All 29 endpoints covered**

Note: 15 screens use fixed routes; 3 additional dynamic-parameter variants (booking detail, business detail, and its action states) bring the practical total to 18 distinct rendered views.

Recommended Build Order

- **1. Auth screens** (Register, Login, Forgot/Reset Password) — nothing else works without these.
- **2. Public booking page** — the highest-value screen in the product; fully independent of the owner dashboard, can be built and demoed standalone.
- **3. Owner dashboard shell + Business Profile** — establishes the authenticated layout, navigation, and API client pattern reused by every subsequent owner screen.
- **4. Availability + Form Builder** — owner setup screens, needed before Bookings has meaningful data to show.
- **5. Owner Bookings (list + detail)** — the owner's main recurring workflow.
- **6. Admin screens** — lowest priority if time constraints arise; the platform is fully functional for owners and customers without this tier.

Public & Customer Screens

PUBLIC / CUSTOMER — 6 screens

Unauthenticated screens. Covers the customer-facing booking flow, plus the account creation and recovery screens every owner also passes through before becoming authenticated.

#	Screen	Route	Purpose	Backend Endpoints	Key Components	Flow Notes
1	**Landing / Marketing**	`/`	Explains the product, links to registration and login. Not required by any use case — optional polish if time allows.	`None`	Hero section, CTA buttons	Entry point only. Can be skipped/minimal without affecting core flows.
2	**Register**	`/register`	Business owner signs up: personal + business details in one form.	`POST /auth/register`	Multi-field form, inline validation, submit button, success state ("pending review")	On success, do NOT log in — show a pending-approval message per UC4. No auto-redirect to dashboard.
3	**Login**	`/login`	Shared login for owner and admin — role is read from the JWT after login, not chosen on the form.	`POST /auth/login`	Email/password form, "forgot password" link, error banner	On success, redirect based on decoded role: admin to the admin overview, owner to the dashboard. On 403 (pending/rejected/suspended), show the exact message from the response, not a generic error.
4	**Forgot Password**	`/forgot-password`	Customer requests a reset link by email.	`POST /auth/forgot-password`	Single email field, submit	Always show the identical success message regardless of outcome — never let the UI hint whether the email exists.
5	**Reset Password**	`/reset-password?token=...`	Owner sets a new password using the token from their email.	`POST /auth/reset-password`	Token read from URL query param, new-password field, confirm field	On invalid/expired token, show a clear message and a link back to Forgot Password. On success, redirect to Login, not auto-login.

#	Screen	Route	Purpose	Backend Endpoints	Key Components	Flow Notes
6	**Public Business Booking Page**	`/book/[businessId]`	The core customer-facing flow: view business info, pick a date, pick a time, fill the dynamic form, submit.	`GET /public/businesses/:id` `GET /public/slots` `POST /public/bookings`	Business header (name/description/phone), date picker, slot grid (available/unavailable), dynamic form renderer (reads fieldType per field), submit button, confirmation state	This is the highest-value screen in the whole product. Slots must re-fetch on date change. On 409 (slot taken), re-fetch slots and prompt re-selection rather than a dead-end error. Form fields are rendered dynamically from the business's own field definitions — no hardcoded field list.

Business Owner Screens

BUSINESS OWNER — 6 screens

Authenticated screens behind `authenticate` + `requireApprovedBusiness`. Covers registration outcome, profile management, availability setup, form building, and the booking approval workflow.

#	Screen	Route	Purpose	Backend Endpoints	Key Components	Flow Notes
7	**Dashboard Home**	`/dashboard`	Landing screen after owner login — quick overview and entry point to every other owner screen.	`GET /owner/business` `GET /owner/bookings`	Nav sidebar (Profile / Availability / Form / Bookings), summary cards (pending bookings count, business status), recent bookings preview	If the business is pending/rejected/suspended, the API 403s on every owner call — show a full-page status message instead of a broken dashboard, not a per-widget error.
8	**Business Profile**	`/dashboard/profile`	View and edit business details.	`GET /owner/business` `PATCH /owner/business`	Read-only fields (name, slug — shown but disabled with a tooltip explaining they're permanent), editable fields (description, phone, contactEmail, websiteUrl), save button	Client must never send name/slug in the update — disable those inputs entirely, don't just show a validation error after submit.
9	**Availability Settings**	`/dashboard/availability`	Configure the weekly recurring schedule — the input data slot calculation reads from.	`GET /owner/availability` `PUT /owner/availability`	7-row weekly grid (Mon–Sun), per-day toggle (open/closed), start/end time pickers, optional break start/end, slot duration selector, save button	Always submit all 7 days in one request, even unchanged ones — the API requires the complete week every time. Validate client-side before submit (end after start, break within window) to avoid round-tripping obvious errors.

#	Screen	Route	Purpose	Backend Endpoints	Key Components	Flow Notes
10	**Form Builder**	`/dashboard/form`	Customize the dynamic booking form beyond the default Name/Email/Phone fields.	`GET /owner/form` `PUT /owner/form` `POST /owner/form/fields` `PATCH /owner/form/fields/:id` `DELETE /owner/form/fields/:id` `PUT /owner/form/fields/reorder`	Form title/description editor, field list (drag-to-reorder), add-field button, per-field edit/delete icons, field-type selector (text/textarea/dropdown/checkbox/radio), options editor	Protected fields (Name, Email) must show as visually locked — no edit/delete icons at all, not icons that error on click. Reorder should submit the full list on drag-end, matching the all-or-nothing contract.
11	**Bookings List**	`/dashboard/bookings`	Browse and filter all bookings for the business.	`GET /owner/bookings`	Status filter tabs (All/Pending/Approved/Rejected/Cancelled), search box, paginated table (customer, date, time, status), row click to open detail	Pending bookings are the primary action item — consider defaulting the view to "Pending" or visually emphasizing them, since approving bookings is the owner's main recurring task.
12	**Booking Detail**	`/dashboard/bookings/[id]`	View one booking's full submitted answers and take an action on it.	`GET /owner/bookings/:id` `PATCH /owner/bookings/:id/approve` `PATCH /owner/bookings/:id/reject` `PATCH /owner/bookings/:id/cancel`	Customer info block, submitted field answers (label/value pairs), status badge, owner notes textarea, action buttons (approve/reject shown only when pending; cancel shown only when approved)	Only show the action buttons that are actually valid for the current status — don't show "Approve" on an already-approved booking and rely on the API to reject it.

Platform Admin Screens

PLATFORM ADMIN — 3 screens

Authenticated screens behind `authenticate` + `requireAdmin`. Covers registration approval, platform overview, and business listing/suspension. Lowest priority tier — the platform is fully functional for owners and customers without it.

#	Screen	Route	Purpose	Backend Endpoints	Key Components	Flow Notes
13	**Admin Overview**	`/admin`	Landing screen after admin login — platform-wide health snapshot.	`GET /admin/stats`	Stat cards (total businesses, pending registrations, approved/rejected/suspended counts, lifetime bookings, bookings this week), nav to business list	This is the one screen whose data is deliberately not tenant-scoped — it's meant to show everything, across every business, at once.
14	**Business List**	`/admin/businesses`	Browse every business on the platform, across all tenants.	`GET /admin/businesses`	Status filter, active/inactive filter, search box, paginated table (business name, owner, status, booking count), row click to open detail	This is the most significant screen for demonstrating role-based access control — an admin genuinely sees every tenant's row here, unlike every owner screen, which only ever sees its own.
15	**Business Detail**	`/admin/businesses/[id]`	View one business's full detail and take a platform-level action on it.	`GET /admin/businesses/:id` `PATCH /admin/businesses/:id/approve` `PATCH /admin/businesses/:id/reject` `PATCH /admin/businesses/:id/suspend` `PATCH /admin/businesses/:id/activate`	Business + owner info block, status badge, booking count, rejection-reason input (shown only when rejecting), action buttons (approve/reject shown only when pending; suspend/activate shown only when approved)	Same rule as the owner booking-detail screen: only show buttons valid for the business's current state. Rejecting requires a reason — don't let the button submit without one.

Architecture & Interaction Decisions

The same way the backend's conventions keep every route consistent, these decisions are made once and followed by every screen above — not re-decided page by page. Each was reasoned through against what the backend already does, rather than adopted as a generic default.

1 Data fetching and state

Decision. Server Components handle initial page loads with a direct `fetch` call in the component body — no `useEffect`, no client-side loading state for first paint. Client Components (marked `"use client"`) handle interactive pieces only: buttons, forms, anything needing `useState` or `onClick`. After a successful mutation, call `router.refresh()` rather than manually patching local state.

Deliberately avoids adding a data-fetching library (React Query, SWR) on top of an already-new framework — that would be two new systems to learn simultaneously, for a caching benefit this application's scale doesn't need.

`router.refresh()` re-runs the server-side fetch and swaps in fresh HTML without a full page reload. Letting the server re-derive the truth is safer than manually rewriting local state, which risks the UI drifting out of sync with the database.

`fetch` is used throughout rather than axios — it is a web standard available in both the browser and Node, and Next's own caching behaviour is built around it specifically.

2 Session expiry handling

Decision. A single global check inside the shared fetch wrapper: any `401` clears the stored token and hard-redirects to the login screen with a message explaining the session expired. A `403` carrying a business-status message is not treated as an expiry — that message is shown inline on the current screen instead.

Tokens last 24 hours with no refresh mechanism, so expiry mid-session is a realistic scenario, not an edge case. Without a centralised handler the dashboard silently breaks with confusing errors and no path back in.

The 401/403 distinction matters: `requireApprovedBusiness` returns 403 with real, user-facing messages (pending review, not approved, account suspended) that must be displayed, not redirected past. Only a genuinely invalid or expired token warrants the redirect.

A hard redirect is correct here rather than client-side navigation — the session itself is invalid, so discarding all client state is the intended outcome.

3 Loading, empty, and error states

Decision. Three distinct states, handled three distinct ways. Loading uses Next's `loading.tsx` convention per route. Empty uses a shared `EmptyState` component with a purposeful message. Error uses a shared `ErrorState` component with a retry action. All three live in the page's content area, not as transient notifications.

An empty list is not an error and must not look like one. A brand-new business genuinely has zero bookings — rendering blank space makes a correct response look like a broken screen. One line of copy is the entire fix.

Distinguishing empty from error also matters: a network failure and "you have no bookings yet" must not look identical, or the user cannot tell whether to retry, wait, or ignore it.

Highest priority screens for this treatment: the owner bookings list (guaranteed empty on day one) and the public slots grid (a fully-booked or closed day produces an empty-looking grid that needs explicit explanation).

4 Client-side validation scope

Decision. Only checks that are free or trivially cheap: native HTML validation (`required`, `type="email"`), password-confirmation matching, and the password-strength rule mirrored from the backend's own regex. Everything with real logic behind it submits to the server, and the returned error message is displayed as-is.

Duplicating real validation logic creates two copies that must agree forever. If the backend rule changes, the frontend copy silently goes stale and starts misleading users — worse than having no client-side check at all.

This is not a security compromise. The backend was always the sole authority: booking submission re-derives slot availability from scratch on every request rather than trusting anything the client claimed earlier. Client-side validation is a courtesy for responsiveness, never a boundary.

The password-strength regex is a deliberate exception — a fixed rule that does not depend on live data, safe to mirror for instant typing feedback, with the server still having the final word on submit.

Where the valid options list is already in memory (dropdown, radio, and checkbox fields fetched with the form definition), checking against it client-side is free and carries no drift risk — that is reading the same data, not duplicating logic.

5 Notifications and error surfacing

Decision. Toast notifications for action outcomes — a booking approved, availability saved, a profile updated — appearing briefly and fading. Persistent inline messages for form validation errors and failed page loads, shown next to the relevant field or above the submit control.

A toast is right for "something just happened" but wrong for "something is wrong with what you are filling in." A validation message that fades after three seconds leaves the user guessing what to fix.

The backend already writes complete, human-readable error strings for every failure case. The frontend displays those directly rather than maintaining its own parallel copy of error text.

Toast component comes from shadcn/ui rather than being hand-built, styled once to the project palette.

6 Embeddable booking widget

Decision. A popup-modal embed delivered via a standalone `embed.js` file written in plain JavaScript. The business owner copies a single script tag carrying their business identifier — not an iframe tag. The script injects a trigger button and opens the booking page inside a modal overlay whose dimensions the script controls.

A plain inline iframe is confined to whatever space the host page's layout allows, so the booking interface gets squeezed by the business's own sidebar and content columns. A script-controlled overlay renders above the host page entirely, so its size is not negotiated with someone else's design.

The owner's experience is unchanged in effort — still a single copy-paste — but simpler in practice: a script tag can be pasted anywhere on the page, whereas an iframe must be positioned deliberately and sized manually.

Written in vanilla JavaScript rather than React, since it must run on arbitrary third-party websites that may not use React at all. Served from the application's public directory at a stable URL.

Cross-origin behaviour cannot be meaningfully verified against a same-origin local test — frame-blocking headers only trigger when the embedding page is genuinely a different origin. Verification will use a tunnelling tool to expose the local development server at a temporary public URL, embedded on a real, separately-hosted website.

7 Responsive scope

Decision. The public booking page must be genuinely usable on mobile. Owner dashboard and admin screens are desktop-first and mobile-tolerable: they must not break, but no dedicated mobile layout is required.

Booking links are shared by text message, social media, and printed material — mobile access is the realistic default for a customer, not an edge case.

An owner reviewing and approving bookings, or an administrator reading platform statistics, is far more likely to be at a desktop. Full responsive polish across every screen is real effort with limited return, so it is concentrated where mobile access is actually expected.

8 Styling and component library

Decision. Tailwind CSS for styling, with the palette, spacing scale, and sizing defined once in the theme configuration as the single source of truth. shadcn/ui for components — copied as editable source files into the project rather than installed as an opaque dependency.

shadcn/ui components carry no visual identity of their own. They are written against Tailwind theme variables, so defining the palette once in configuration restyles every component consistently and automatically — unlike libraries such as Material UI, whose own design opinions must be actively overridden.

Because the component source lives in the project rather than in a package directory, any default that does not suit can be edited directly, with no risk of an update overwriting the change.

This also delivers the consistency guarantee that matters most: heading sizes, colours, spacing, and border radii come from one configuration file referenced everywhere, rather than being chosen ad hoc per screen.

9 Session storage and transport

Decision. The existing JWT is stored in an `httpOnly` cookie, set by a Next.js route handler after it calls the existing login endpoint. The backend is unchanged. All browser traffic goes to the Next.js origin only — Server Components fetch reads directly, and a single catch-all proxy route forwards mutations. User identity is read server-side in each route group's layout and passed down as props.

This resolves a direct conflict with decision 1. Server Components run on the server, where no browser exists, so they cannot read local storage — meaning a token kept there makes server-side authenticated fetching impossible. Cookies are transmitted automatically with every request, including the request for the page itself, so server-side code can read them.

Having Next.js set the cookie rather than the backend avoids cross-domain cookie complications between two separate deployment platforms. The backend continues returning the token in its response body exactly as it does today and never knows a cookie exists — no backend changes at all.

`httpOnly` means JavaScript cannot read the token, removing the cross-site-scripting theft risk that local storage carries. The tradeoff is that automatically-transmitted cookies introduce cross-site request forgery exposure, mitigated by setting `SameSite` explicitly rather than relying on browser defaults.

Two consequences follow. Because JavaScript cannot read the token, Client Components cannot call the API directly — hence the proxy route, written once and covering every endpoint. And logout must be a server call that expires the cookie, since JavaScript cannot clear an `httpOnly` cookie.

The `Secure` flag is applied in production only; it restricts cookies to HTTPS and would silently fail during local development.

User identity is passed down as props rather than duplicated into a second readable cookie: one source of truth, no synchronisation to maintain, and any genuine security decision must happen server-side regardless.

10 Route protection

Decision. Session verification happens inside each route group's layout, which is a Server Component. Not in middleware.

The layout already reads the session to pass user identity downward, so the guard costs nothing extra — one read serving two purposes.

Next.js middleware runs in a restricted runtime where the backend's existing JWT library does not work. Using middleware would mean adding a second, different verification library purely for that layer — two code paths verifying the same token, which is exactly the kind of duplication that drifts apart over time.

Because the check runs on the server before any HTML is sent, there is no flash of protected content followed by a redirect — the familiar problem in client-rendered applications, where the check can only run after the page has already painted.

Worth being clear-eyed about what this guard is for: it is a user-experience feature, not the security boundary. Every protected endpoint independently verifies the token and checks the role server-side, already built and verified. Bypassing the frontend guard would reveal nothing.

11 API address configuration

Decision. Resolved as a consequence of decision 9 rather than as an independent choice. The backend's address is a server-side environment variable, read only by Server Components and the proxy route, and never exposed to the browser.

Because the browser only ever communicates with the Next.js origin, there is no public API address for client code to know or configure.

12 Cross-origin request handling

Decision. Also resolved as a consequence of decision 9. The browser-to-backend cross-origin path is removed from the architecture entirely.

All browser requests target the Next.js origin, and Next.js calls the backend server-to-server, where cross-origin restrictions do not apply. This removes what would otherwise have been the least-tested part of the integration — every endpoint so far has been verified through a REST client, which does not enforce cross-origin rules at all.

13 Cross-role navigation

Decision. Silent redirect to the user's own home area. No session redirects to login; a valid session on the wrong role's routes redirects to that role's own landing screen.

For an authenticated user this is almost always accidental — a stale bookmark, a mistyped address, or a shared link. Redirecting is lower friction than an access-denied screen, which suits situations where someone might reasonably have expected access. Here the two roles are cleanly separated, so there is no ambiguity to explain.

14 Date and time handling

Decision. Dates and times are treated as strings for display and never parsed into date objects for rendering. Date objects are used only where arithmetic is genuinely required, and always through UTC methods.

JavaScript's date object applies the browser's local timezone automatically. Passing a time through it for display means a customer in a different timezone could see an entirely different hour than the business configured. The API already returns display-ready strings, so no parsing is needed.

Where arithmetic is unavoidable — the date picker establishing today's date, or bounding the booking window — using UTC methods throughout mirrors the discipline the backend already follows in its own slot calculation.

Known limitation, carried forward deliberately: the application has no timezone concept at all, so a business and its customers are assumed to share one. Appropriate for locally-operating service businesses; a genuine constraint if the product ever expands beyond that.

15 Form state management

Decision. A form library for the two genuinely complex forms — the weekly availability grid and the form builder. Plain component state for simple forms such as login and password reset.

A two-field form gains nothing from a library. The availability grid is a different matter: seven rows, each with a toggle plus five conditional fields, with validation spanning multiple fields at once. Managing that as hand-maintained nested state is where forms genuinely become painful.

The chosen component library's form primitives are already built around this form library, so adopting it deliberately for the complex cases introduces less new surface than it first appears.

Mixing approaches is intentional rather than inconsistent — the goal is the least total complexity, not uniformity for its own sake.

Decision Summary

All fifteen decisions in one place for quick reference. Every question identified during planning has been resolved; none remain open.

#	Area	Decision
1	**Data fetching**	Server Components for initial loads; `fetch` throughout; refresh the server render after mutations
2	**Session expiry**	Global check in the shared fetch wrapper; expired session redirects to login, business-status blocks shown inline
3	**Screen states**	Route-level loading files; shared empty and error components in the content area
4	**Validation**	Free and structural checks only on the client; everything with real logic round-trips to the server
5	**Notifications**	Toasts for action outcomes; persistent inline messages for validation and failed loads
6	**Embed widget**	Popup modal via a standalone script the owner pastes in; verified cross-origin through a tunnel
7	**Responsive scope**	Public booking page mobile-usable; owner and admin desktop-first, mobile-tolerable
8	**Styling**	Tailwind with a single theme configuration; shadcn/ui components copied in as editable source
9	**Session storage**	JWT in an `httpOnly` cookie set by the frontend; all traffic proxied through Next.js; no backend changes
10	**Route protection**	Verified in each route group's layout, server-side, not in middleware
11	**API address**	Server-side environment variable only; never exposed to the browser
12	**Cross-origin**	Eliminated — the browser only ever talks to the frontend origin
13	**Cross-role access**	Silent redirect to the user's own landing screen
14	**Dates and times**	Strings for display; date objects only for arithmetic, always in UTC
15	**Form state**	Form library for the availability grid and form builder; plain state elsewhere

Next Step

One artifact remains before scaffolding begins: a design specification fixing the palette, typography scale, spacing, and component states in detail — the layer these decisions reference but do not define.